

Cognitive Operations and Reliability Engine (CORE)

Industrial Knowledge Intelligence & Unified Asset Brain

[Live Prototype](#) • [GitHub Repository](#) • [Demo Video](#)

1. Problem Statement & Market Gap

The asset-intensive industrial sector operates on highly fragmented documentation, including engineering drawings, process flow diagrams (PFDs), PFMEAs, control plans, and inspection reports. This knowledge is typically scattered across **7 to 12 disconnected** digital and physical systems within a single large plant. The barrier to operational excellence is not a lack of data, but the inability to reason across these disconnected silos. Three critical gaps define this market:

- **The Search Tax:** Professionals working in asset-intensive industries waste an average of **35% of their working hours** simply searching for information, recreating existing documents, or clarifying lost instructions.
- **Fragmentation-Driven Downtime:** Maintenance and quality engineers are forced to make decisions without the complete equipment history or context regarding failure patterns. This data fragmentation directly contributes to **18–22% of unplanned downtime** events in the Indian heavy-industry sector.
- **The Knowledge Cliff:** Within the next decade, an estimated **25% of India's experienced** industrial operators and engineers will retire, taking decades of undocumented, tacit operational knowledge with them, which cannot be recovered once lost.

The Mandate: The industrial sector requires a shift from disjointed file storage to a unified, autonomous intelligence layer. The system must treat disparate documents—like drawings, PFDs, and control plans—not as separate files, but as **interconnected views of the exact same operation**, joined by a shared operation number and characteristic spine.

2. CORE: Multi-Agent Knowledge Infrastructure

CORE is a fully autonomous, **nine-agent platform** designed to ingest heterogeneous industrial documents and construct a unified knowledge base. By combining an ontological Knowledge Graph with a RAG vector store, the system makes enterprise data queryable, auditable, and continuously updated at the point of need.

The 6-Phase Execution Pipeline

1. **Ingest & Classify:** A document is uploaded and its type is identified using a deterministic two-stage classifier. Deterministic rules (filename or sheet-name heuristics) are applied first, and an LLM is only invoked if the rules are unsure, after which it routes the document to the correct parser.
2. **Extract:** Type-specific parsers extract structured entities and relationships, successfully navigating the deep, merged-cell layouts typical of industrial PFMEA and control-plan spreadsheets. Data points such as operations, characteristics, specifications, and failure modes (with S/O/D/RPN scores) are pulled accurately.
3. **Build the Graph:** Extracted entities are resolved against a shared “operation + characteristic” spine and written into the Knowledge Graph. Every single node carries a **source citation** mapping it back to the exact document, sheet, and row.
4. **Embed for RAG:** Every ingested record is simultaneously embedded into a vector store. This dual architecture lets free-text natural-language questions retrieve contextual evidence across multiple parts and document types via semantic search.
5. **Audit on Ingest:** The moment new data arrives, a deterministic rules engine runs **C1–C10** consistency checks over the graph, autonomously detecting cross-document quality gaps and compliance violations before they escalate.
6. **Answer & Act:** The Copilot Orchestrator acts as the conversational broker, using tool-calling to route queries to specialist agents (RCA, QMS, Lessons Learned). These agents turn the knowledge base into root-cause analyses, evidence packs, and cited answers delivered directly to field technicians.

3. Innovation & Differentiation

- **The APQP Digital Thread:** CORE is the first platform to automatically stitch the entire product lifecycle (Drawing → PFD → PFMEA → Control Plan → Inspection → NCR → CAPA) into one traceable graph. Traceability that quality engineers build manually becomes entirely automatic.
- **Deterministic Consistency Auditor:** Ten strict, rule-based checks (C1–C10) run directly over the graph structure. Because there is no LLM guessing in the decision path, gap detection—including finding out-of-specification measurements from raw inspection data—is a **mathematical, evidence-cited guarantee**.
- **Systemic Failure Intelligence:** The platform surfaces operational patterns invisible to single-document reviews, such as a single root cause driving high-RPN failures across five different operations, or a recurring defect theme spanning multiple parts.
- **Agentic, Not Assistive:** The Copilot is an autonomous tool-calling orchestrator that independently selects which specialist agent to invoke (graph traversal, RAG, RCA, QMS) rather than acting as a simple prompt wrapper over a text blob.
- **Citations as a First-Class Contract:** Every node stores its origin, so every finding and answer explicitly links back to the exact source record—transforming the tool from an advisory

- chatbot into enterprise-grade decision support.
- **Graceful Degradation:** The platform is engineered to run completely offline. If API keys are removed, it falls back to

lexical retrieval and rule-based agents, ensuring air-gapped plants maintain operational intelligence.

4. System Architecture

CORE relies on a layered architecture enforcing strict data flows from ingestion through extraction, graph construction, and reasoning.

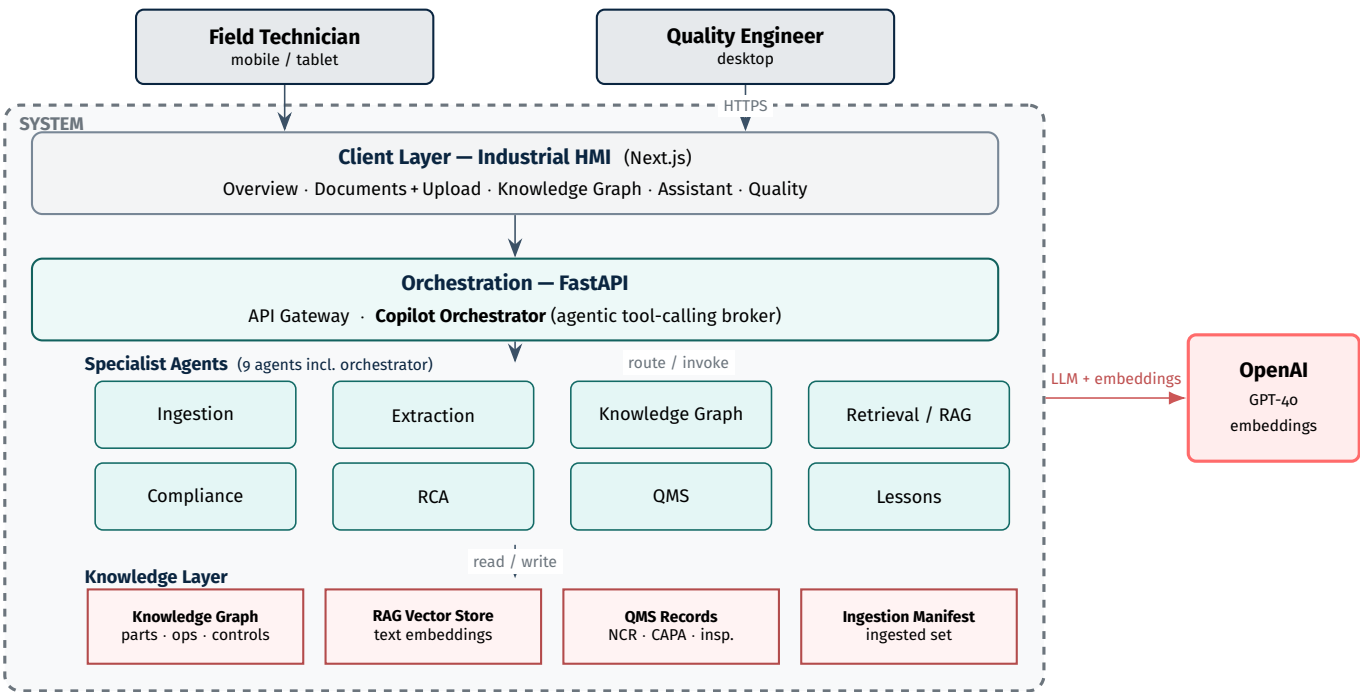


Figure 1: System architecture — users, the system boundary with its client, orchestration, nine-agent and knowledge layers, and the external LLM service.

Layer Summary

Layer	Components & Functions
Client Layer (HMI)	Mobile-first, shop-floor-legible Next.js UI. Features Overview, Document Upload, Graph visualisation, and specialised Quality dashboards (Gaps, QMS, RCA).
Orchestration Layer	FastAPI gateway housing the Copilot Orchestrator, which acts as the exclusive tool-calling broker across the multi-agent system.
Intelligence Agents	Ingestion, Extraction, Knowledge Graph, Retrieval/RAG, and Lessons Learned / Failure Intelligence agents.
Governance Agents	Compliance & Gap Detection (deterministic C1–C10 rules), RCA Agent, and QMS Agent.
Knowledge & State Layer	Unified Knowledge Graph, Vector Store (NumPy / Qdrant), and persisted QMS records.

The 9-Agent Ecosystem

1. **Ingestion Agent:** Accepts and classifies incoming files (PDFs, spreadsheets), routing them via heuristics or fallback LLMs.
2. **Extraction Agent:** Parses merged-cell industrial matrices to isolate entities.
3. **Knowledge Graph Agent:** Traverses the unified graph, resolving parts, operations, and controls to serve complete sub-graphs for specific queries.
4. **Retrieval / RAG Agent:** Executes semantic search across the vector store using LLM embeddings or an offline BM25-style fallback.
5. **Copilot Orchestrator:** The central nervous system that runs the agentic loop, invoking specialists and compiling cited responses.
6. **Compliance & Gap Detection Agent:** Executes the C1–C10 deterministic checks (e.g., verifying whether a drawing tolerance matches the control plan).
7. **Maintenance / RCA Agent:** Fuses failure histories, 8D CAPAs, and inspection read-outs to recommend corrective and preventive actions.
8. **Quality / QMS Agent:** Assembles audit-ready compliance evidence packs and links open non-conformances to specific parts.
9. **Lessons Learned Agent:** Mines historical data across all parts to push proactive, cross-part warnings regarding systemic defects.

Data Flow

The end-to-end data flow moves a document from upload through classification, extraction and graph/vector storage, then branches into the on-ingest consistency audit and the copilot query path—where a natural-language question is routed to specialist agents and answered with evidence links back to the source record.

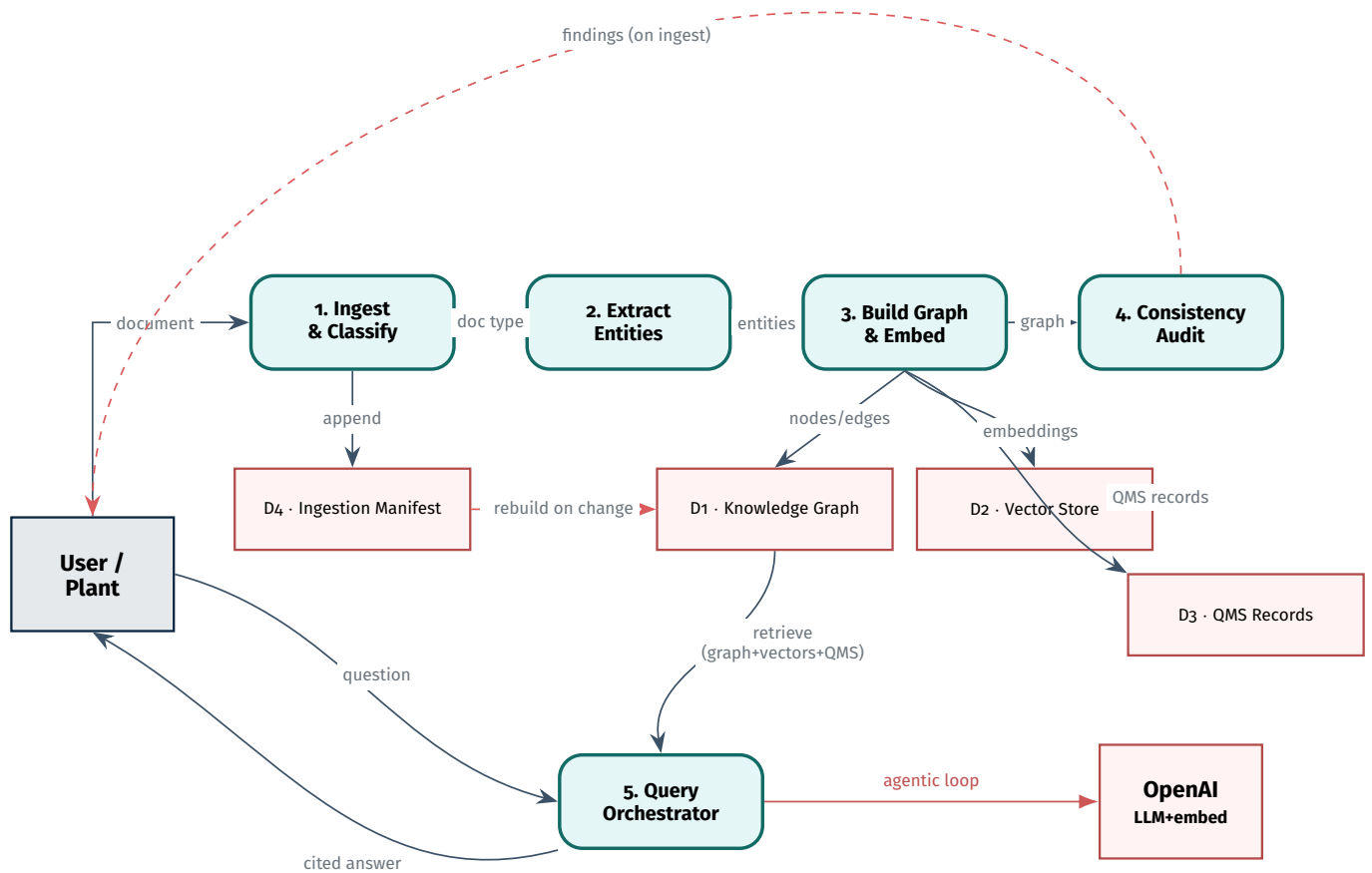


Figure 2: Data flow — ingestion pipeline (1–4) with its data stores, the on-ingest audit, and the copilot query path (5) with its agentic tool-calling loop.

5. Engineering Decisions & Technical Depth

- **Graph-in-Context for Reliability:** For individual parts, the entire structured graph is small enough to fit within an LLM context window. Structural questions are answered directly from the graph without fragile query translation, reserving standard RAG for free-text narratives.
- **Deterministic Logic, LLM Phrasing:** Every compliance violation is mathematically decided by explicit rules (C1–C10) running over the graph. The LLM is strictly restricted to phrasing findings in natural language and is never allowed to decide whether a part is compliant.
- **Out-of-Spec Detection from Raw Readings:** The system parses raw numbers from inspection observations and compares them against authoritative tolerances (matching the control-plan limit), transforming static reports into live non-conformance alerts.
- **Incremental, Persisted Knowledge:** The database is driven by an ingestion manifest. Appending a document rebuilds the graph and embeds only new records using content-hash caching, allowing the system to grow live and reset cleanly.
- **Provider-Agnostic Interfaces:** Document parsers sit behind clean abstraction layers, ensuring future upgrades—such as a managed Neo4j graph or a computer-vision model—can be hot-swapped without altering downstream code.

Tech Stack	Backend & Orchestration	Python, FastAPI, 9-Agent System, tool-calling orchestration
	AI & Semantic Layer	OpenAI (GPT-4o, text-embedding-3-small), RAG vector store
	Knowledge & Extraction	openpyxl, xldr, pdfplumber, pypdfium2, Knowledge Graph
	Frontend (Industrial HMI)	Next.js, TypeScript, Tailwind CSS, react-force-graph
	Data & State	NumPy vector store, Chroma / Qdrant, persisted ingestion manifest

6. Business Impact & Value Proposition

Metric	Fragmented Status Quo	With CORE Platform
Cross-Document Query Time	Hours to days of manual reconciliation	Seconds, with verified citations
Systems to Consult	7 to 12 disconnected repositories	1 unified brain
Time Lost to Search	~35% of working hours	Sharply reduced
Quality-Gap Detection	Manual, sampled, after the fact	Automatic upon document ingest
Fragmentation Downtime	Drives 18–22% of unplanned events	Directly targeted and minimised
Retiring Expert Knowledge	Lost forever (25% retire in a decade)	Captured as a living graph

Impact Model

CORE does not simply digitise documents; it algorithmically connects them. Using the C10 check, the system flagged a base-casting weight measuring **97.9 g against a 98–104 g tolerance**—a nonconformance that had previously escaped detection as a static data point. It surfaced a systemic root cause (*“Coolant concentration not as per standard”*) driving failures across **five distinct operations**, none of which had a recommended action.

7. Technical Challenges & Roadmap to Plant Scale

Anticipated Technical Challenges

Heterogeneous, Messy Formats: Real-world PFMEA and control-plan spreadsheets use incredibly complex, deep merged-cell layouts. Robust entity extraction from these grids is the most difficult technical hurdle and where system accuracy is truly won.

Drawing Digitisation Limitations: Because vector-CAD drawings encode physical dimensions as graphics rather than text, reliable extraction requires dedicated computer-vision pipelines, not simple text parsing.

Entity Resolution at Scale: A single characteristic named slightly differently across dozens of documents and parts must be mathematically reconciled into a single graph node without triggering false merges.

Scale of Graph and Vectors: Scaling to thousands of distinct manufacturing parts will require migrating from in-memory processing to managed enterprise graphs and delta-embedding strategies.

Scalability Roadmap

Computer Vision & OCR Ballooning: A dedicated **YOLO** balloon-detection and OCR model trained for high-accuracy engineering-drawing digitisation, enabling the C1 tolerance check to run at enterprise scale.

Managed Infrastructure: Swap the current NumPy / in-memory interfaces for production-grade **Neo4j** and **Qdrant** instances to handle plant- and enterprise-wide document corpora.

Regulatory Corpus Integration: Ingest external statutory norms (IATF 16949, ISO 9001, Factory Act) and automatically audit the plant’s active procedures against these standards.

Expanded Telemetry & Tacit Knowledge: Introduce ingestion pipelines for unstructured email archives, incident records, and work orders to map the complete operational history—ultimately integrating live SCADA/IoT operating conditions into the RCA agent for real-time predictive failure intelligence.

8. Deliverables & Links

Working Prototype — Live Deployment

<https://core-et.vercel.app>

The platform, deployed and running. It boots to an empty knowledge base by design — upload documents on the **Documents** tab and watch the graph, quality gaps, QMS views and copilot populate live.

Source Code — GitHub Repository

https://github.com/advaitithape/CORE_ET

Complete platform source: the nine agents, knowledge-base and RAG layer, document parsers, FastAPI backend, industrial HMI frontend, and one-command deployment configurations (Docker / Render / Vercel). Code-only by design — no plant documents are distributed.

Demo Video

drive.google.com/file/d/14Wz58dz83FJYXAgA36nVTL8yGFQENiNX/view

A guided walk-through: clean start → document ingestion → knowledge graph → assistant with cited answers → automated quality gap detection, QMS, RCA and cross-part lessons.

From fragmented documents to one governed, queryable, self-auditing **Cognitive Operations & Reliability Engine**.